Two-Person Speaking Script

Self-Supervised AI for ETL Error Detection and Root-Cause Localization

- Before you start
- Slide 1: Title
- Slide 2: The problem
- Slide 3: Two reasons this is hard
- Slide 4: One fault, nine suspects
- Slide 5: There was no dataset
- Slide 6: The pipeline
- Slide 7: Fifteen ways it breaks
- Slide 8: Measuring the damage
- Slide 9: How the model learns
- Slide 10: Finding the culprit
- Slide 11: Results
- Slide 12: The main finding
- Slide 13: What didn't work
- Slide 14: The surprise
- Slide 15: Explaining the answer
- Slide 16: Limitations
- Slide 17: What next
- Slide 18: Close, then demo
- Question time
    - Questions X should be ready for
    - Questions Akshat should be ready for
- Timing

For a 15 minute presentation with roughly 5 minutes of questions.

Akshat presents the problem, the method, the results and the demo. **X** presents the dataset and simulator, slides 5 to 8, about 3 minutes.

Replace **X** throughout with the actual name before printing.

Before you start

Whoever speaks a section has to be able to answer questions on it. Examiners often direct a question at whoever just spoke. X should read Part 3 of `docs/EXPLAINER.md` and be comfortable with the three questions listed at the end of his section. That is a short read and it is the whole preparation he needs.

Agree the handover out loud in rehearsal. Most two-person presentations fall apart at the swap, not in the content. Say the handover line exactly as written so the other person knows their cue.

One laptop, one driver. Akshat drives the slides throughout, including during X's section. Swapping the clicker mid-talk wastes ten seconds and looks disorganised.

Slide 1: Title

AKSHAT *(about 20 seconds)*

> Good morning. Our project is called Self-Supervised AI for ETL Error Detection and Root-Cause Localization.
>
> In plain terms: companies move data through pipelines, those pipelines break quietly, and when they break every step downstream looks broken too. We built something that finds the one step that actually caused it, without needing labelled examples of failures.
>
> I'll take the problem and the method, and X will take the dataset we built.

Slide 2: The problem

AKSHAT *(about 60 seconds)*

> Here's the failure we care about.
>
> When people imagine a pipeline breaking, they imagine a job crashing. Somebody gets an alert, it gets fixed the same day. That's the easy case and it's already solved.
>
> What actually happens is on the right. The job finishes successfully. But upstream, a price field switched from rupees to paise. Every number is now a hundred times too big, and nothing failed, so nothing alerted.
>
> Nobody notices for weeks. Every report built on that table is wrong for the whole period, and it gets found by a human squinting at a dashboard.

Slide 3: Two reasons this is hard

AKSHAT *(about 60 seconds)*

> Two things make this hard to automate.
>
> First, there are no labels. Failures are rare and recorded badly, and a team that has just spent a night fixing one is not going to sit down and annotate it. So the model has to learn what normal looks like entirely on its own.
>
> Second, and this is the interesting one: symptoms are not causes. When one step breaks, every step after it looks broken too. And the damage usually grows as it travels. A small problem at extraction becomes a big problem after aggregation. So the loudest alarm is often the furthest from the real fault.

Slide 4: One fault, nine suspects

AKSHAT *(about 45 seconds)*

> This picture is the whole problem in one image.
>
> A fault was injected into exactly one task. Look what happens: eight of the nine tasks light up, and several of them move harder than the one that caused it.
>
> Every one of those would trip an ordinary alarm. Telling an engineer that nine of nine tasks look unhealthy is not help.
>
> So we couldn't just build a detector. We had to build something that separates the cause from the consequences.

Handover line:

> To test any of that, we needed data that doesn't exist. X built it, so I'll let him explain.

Slide 5: There was no dataset

X *(about 45 seconds)*

> Thanks. So the first problem we hit was that there is no dataset for this.

> The standard datasets in this area are server and sensor readings. They have no pipeline in them at all, so you can't even ask "which step caused this", let alone measure whether you got it right.
>
> The research that does use real pipeline data uses company data that never gets released.
>
> So building the benchmark became part of the work rather than a shortcut around it. What we ended up with is eight thousand simulated pipeline runs, nine tasks each, fifteen kinds of failure, and two hundred and seventy-three labelled incidents.

Slide 6: The pipeline

X *(about 45 seconds)*

> This is the pipeline we simulated. Nine tasks. Three of them pull data out of source systems, two clean it, two join things together, one aggregates, and one loads the final table.
>
> Every one of these does real work on real rows: type casting, removing duplicates, converting currencies, actual joins.
>
> And here's the design decision that matters most. We inject faults into the **rows**, not into the statistics.
>
> If we'd corrupted the summary numbers directly, we'd have had to decide by hand how the damage spreads to the next task, and then the model would just be learning our rules about spreading. That's circular.
>
> By breaking the actual data, the next task genuinely reads broken rows, and the damage spreads by itself.

Slide 7: Fifteen ways it breaks

X *(about 40 seconds)*

> Fifteen failure types, in six families. Columns disappearing, fields going empty, rows duplicated or lost, units changing, timestamps shifting, keys not matching.
>
> These aren't invented. They come from the failure taxonomy published in Google's and Amazon's own data validation systems.
>
> The one at the bottom right is deliberately awkward. The task runs slowly and misses its deadline, but the data it produces is perfectly correct. We included it to check the model doesn't confuse being slow with being wrong, and importantly, it should not spread downstream.

Slide 8: Measuring the damage

X *(about 50 seconds)*

> One more thing about the labels, because this is the part we're most pleased with.
>
> The obvious way to label which tasks an incident affected is to say: the one that broke, plus everything after it. That turns out to be wrong. An extra unused column changes nothing downstream. A slow task still produces correct data.
>
> So instead we run every broken batch a **second time** with the fault removed, keeping everything else identical, right down to the random seeds, and compare the two. A task counts as affected only if its numbers actually moved.
>
> Measured, it's about two and a half tasks per incident. The assumption claims nearly four. So the labels are a measurement rather than a guess, which matters because the model is graded against them.

Handover line:

> That's the data. Akshat will take it from here.

Slide 9: How the model learns

AKSHAT *(about 60 seconds)*

> Thanks X. So, how do you train something with no labels?
>
> You hide part of the input and make the model guess what was hidden. The rest of the input is the answer key.
>
> It's the same idea as covering a word in a sentence and predicting it. To be good at that, you have to actually learn how language works. Here, to be good at filling in a hidden task, you have to learn how the pipeline normally behaves.
>
> Then, once it's good at guessing, whatever it **can't** guess is something unusual. That's the anomaly signal, and no labels were involved anywhere.

Slide 10: Finding the culprit

AKSHAT *(about 75 seconds. This is the key slide, slow down.)*

Now the important part. How do you separate the cause from the consequences?

You ask each task a different question. Not “does this look bad”, but “does this look bad **given what it was fed**”.

A task downstream of a failure produces bad output, but that output follows perfectly from the bad input it was handed. Nothing surprising there. It’s a victim.

The task where it started had perfectly healthy input and produced bad output anyway. That’s where the trouble began.

On the right is something I got wrong at first, and I think it’s worth mentioning. My first version hid only the task being tested. But the model could still see that task’s own downstream steps, which were also broken, and it was quietly reading the answer off them. Once I hid the task and everything downstream of it, accuracy went from sixty-three percent to seventy-seven.

Slide 11: Results

AKSHAT *(about 60 seconds. Do not soften this.)*

Here are the results, on the last quarter of the timeline, which the model never saw during training.

I’ll be straight about this: our model doesn’t win. A simpler method detects slightly better, and another one localizes slightly better. All four leading methods are within measurement error of each other.

The reason is that each task is described by only thirty-eight numbers, with five thousand clean examples to learn from. That’s close to ideal for classical statistical methods, so there wasn’t much headroom.

But what we found instead is more interesting, and it’s the next slide.

Slide 12: The main finding

AKSHAT *(about 75 seconds. This is the one to land.)*

Ranking tasks by how bad they look asks the wrong question. It asks which task has the biggest error. What you want is which task is furthest from **its own** normal, because some tasks are naturally harder to predict than others.

There are two completely different ways to fix that.

One is statistical: compare each task against its own history. The other is structural: ask whether it’s surprising given its inputs, which is the idea I described a moment ago.

They're worth exactly the same. Seventy-seven point three percent each, against fifty-nine with no fix at all. Identical to three decimal places. And doing both only adds another two points.

That eighteen point gain is larger than the entire spread between the seven methods, which is about four points. So on this problem, **how you score matters more than which model you use**.

I'll add that an earlier version of my write-up claimed the structural idea was worth far more on its own. It was being compared against an unfair baseline, and the report carries the correction.

Slide 13: What didn't work

AKSHAT *(about 45 seconds)*

We removed each design idea and measured the damage. Four of them changed nothing.

Telling the model the pipeline shape doesn't help, because it works the shape out from the data on its own. We checked that directly by looking at what it pays attention to.

Giving it a clock doesn't help, because a thirty-two hour window already contains a full day.

Adding contrastive learning cost four times the compute and helped nothing.

And training on perfectly clean data was no better than training on messy data, which confirms the claim that it tolerates realistic history.

Slide 14: The surprise

AKSHAT *(about 50 seconds)*

One thing did change something, and it was the opposite of what I expected.

I trained the model for longer. It got better at its training task, and worse at finding the culprit. Eighty percent down to seventy-six.

The reason is that the model depends on being surprised by broken data. Train it long enough and it gets good at predicting the broken data too, so the surprise disappears.

That's awkward for us, because we chose when to stop training using exactly that measure. It's written up as an open problem rather than hidden: we need a stopping rule that needs no labels but still tracks what we actually care about, and we don't have one.

Slide 15: Explaining the answer

AKSHAT *(about 45 seconds)*

> Two things here.
>
> On the left, the model learned the pipeline structure by itself. Those bright squares are the real connections between tasks, and nobody told it which task feeds which.
>
> On the right, it names the right kind of problem ninety-three percent of the time. A unit change gets reported as a value problem, a timezone shift as a timing problem.
>
> Most papers show an explanation picture and never check whether it's right. Because our simulator knows what each fault should look like, we can actually score it.

Slide 16: Limitations

AKSHAT *(about 45 seconds)*

> The honest list.
>
> The data is simulated, so we chose how often faults happen and how severe they are. The ranking of methods is more trustworthy than any single number.
>
> Two failure types localize badly, both involving row counts. We know exactly why, and I'll come back to it.
>
> Detection turned out to be too easy on this benchmark to separate methods well.
>
> And everything is measured on one pipeline shape.

Slide 17: What next

AKSHAT *(about 40 seconds)*

> Three things.
>
> First, and this is the one I'd do immediately: measure how well the join keys overlap. That's the exact missing piece behind the two failure types we're bad at. A join's row count depends on key overlap, and we don't measure it anywhere, so the model literally doesn't have the information. Our own error analysis predicts adding it should fix them.

> Second, find a better stopping rule.
>
> Third, test against a real pipeline.

Slide 18: Close, then demo

AKSHAT *(about 40 seconds)*

> To summarise: we built a benchmark with measured cause labels, a model that learns without any labels and points at the task that broke, and an evaluation that includes four null results, a bug we found in our own work, and a claim we had to correct.
>
> Let me show you it running.

Then open the demo. Follow the walkthrough in `Presentation_and_Demo_Guide.pdf`, about two minutes.

Question time

Default rule: whoever presented the section answers. If it is about the dataset, the simulator or the fault types, X takes it. Everything else, Akshat.

If X gets a question he cannot answer, the recovery is simply:

> “That’s more Akshat’s side, he’ll have a better answer than me.”

That is completely normal in a two-person presentation and reads far better than guessing.

Questions X should be ready for

“Why simulate instead of using real data?”

> “There’s no public dataset that labels which pipeline step caused a failure. The ones everyone uses are server and sensor readings with no pipeline in them. And real company pipeline data isn’t released. So there was nothing to use.”

“How do you know your fifteen faults are realistic?”

> “They come from the failure taxonomy in Google’s and Amazon’s published data validation systems, so they’re documented real-world failure modes rather than ones we made up. What we did choose is how often they happen and how severe they are, and we list that as a limitation.”

“Why inject into rows rather than the statistics?”

> “Because if we corrupted the statistics we’d have to hand-write how damage spreads downstream, and then the model would just be learning our rules. By corrupting the actual rows, the next task really does read broken data and the damage spreads on its own.”

Questions Akshat should be ready for

Full answers are in `Presentation_and_Demo_Guide.pdf`. The four hardest:

- Why doesn’t your model beat the simple methods?
- What’s the weakest part?
- Did you find anything you didn’t expect?
- How do you know it isn’t memorising?

Timing

Section	Speaker	Time
Slides 1 to 4	Akshat	3 min
Slides 5 to 8	**X**	**3 min**
Slides 9 to 12	Akshat	4.5 min
Slides 13 to 18	Akshat	3.5 min
Demo	Akshat	2 min
Total		**16 min**

If you are running short on time, cut slide 13 (what didn’t work) and slide 16 (limitations) down to one sentence each. Never cut slide 12, which is the actual finding, and never cut the demo.